CUSTOMER ORDER COUNT ANALYSIS

Data Analytics Internship | Veda Technologies

1. Project Overview

This project analyzes the Online Retail dataset to count distinct invoices per customer and identify frequent buyers. SQL is used to aggregate transaction records, rank customers by order frequency, and explore customer purchasing behavior.

2. Objectives

• Count distinct orders for each identified customer.
• Identify the top 10 customers by order frequency.
• Avoid overcounting invoices with multiple product lines.
• Examine ranking ties and overall customer/invoice counts.

3. Dataset and Key Columns

Column	Purpose
InvoiceNo	Invoice or transaction identifier
StockCode	Product identifier
Description	Product description
Quantity	Units purchased
InvoiceDate	Date and time of transaction
UnitPrice	Price per unit
CustomerID	Customer identifier
Country	Country associated with the transaction

4. Tools and SQL Concepts

SQL | COUNT(DISTINCT) | GROUP BY | ORDER BY | LIMIT | Common Table Expressions (CTEs) | Window functions

5. Methodology

1. Import OnlineRetail.csv into a SQL database.
2. Inspect customer and invoice identifiers.
3. Count distinct invoices for each CustomerID.
4. Group by customer and sort order counts in descending order.
5. Retrieve the top 10 customers and check ranking ties.
6. Calculate total distinct customers and completed invoices.

6. Important Data Handling Decisions

A single invoice can have multiple product rows. Counting rows directly can therefore overstate the number of orders. COUNT(DISTINCT InvoiceNo) counts each invoice once within each customer group. Customer records with missing CustomerID are excluded from customer-level summaries.

In the provided query approach, invoice numbers beginning with 'C' are treated as cancellation records and excluded from completed-order counts. This is an explicit analysis choice; results will differ if cancellations are included.

7. Expected Deliverables

• Customer order-count table.
• Top 10 frequent customers.
• SQL queries for total customers and completed invoices.
• Ranking tie analysis.

• SQL source file: Customer_Order_Count_OnlineRetail.sql.

8. Key Learning Outcomes

• Count unique transactions accurately.
• Aggregate data by customer using GROUP BY.
• Rank customers using ORDER BY and window functions.
• Recognize duplicate invoice lines and cancellation records.
• Distinguish customer count from order count.

9. Conclusion

This project demonstrates how SQL can convert transaction-level retail records into customer-level insights. Counting distinct invoices avoids double-counting orders with multiple products and helps identify frequent buyers for further customer behavior analysis.

Reporting note: This report documents the analysis method and SQL deliverables. Actual customer rankings and numeric findings are not stated because the SQL queries have not been executed against your imported database in this report.